VITAS. FOOTBALL INTELLIGENCE

PROPHET HORIZON TECHNOLOGY · BUILD 2.1.0 · ABRIL 2026

VITAS · Flujo de Trabajo

> Documento que describe todos los flujos de usuario en la plataforma VITAS. Ultima actualizacion: Abril 2026

Indice

1. Onboarding y Autenticacion
2. Gestion de Jugadores
3. Pipeline de Video Analysis (VitasLab)
4. Tracking en Tiempo Real (YOLO)
5. Reporte de Inteligencia (Claude AI)
6. Comparacion con Profesionales
7. Perfil de Rol Tactico
8. Registro de Eventos de Partido
9. Dashboard y Monitoreo
10. Master Dashboard (Academia)
11. Director Dashboard
12. Billing y Suscripciones
13. Notificaciones Push
14. Knowledge Base (RAG)
15. Sincronizacion y Offline

© 2026 Prophet Horizon Technology

16. Exportacion PDF

1. Onboarding y Autenticacion

```
[Usuario nuevo] ──> /register
      |
      v
  Email + Password ──> Supabase Auth
      |
      v
  Trigger: handle_new_user() ──> Crea perfil en tabla `profiles`
      |
      v
  /onboarding ──> Nombre academia, rol (scout/entrenador/director)
      |
      v
  /pulse (Dashboard principal)
```

Login posterior:

```
/login ──> Supabase Auth
  |
  v
  Session activa ──> syncFromSupabase()
  |                  (descarga jugadores, videos, suscripcion)
  v
  /pulse
```

Modo Offline (sin Supabase configurado):

```
App inicia ──> detecta VITE_SUPABASE_URL ausente
  |
```

© 2026 Prophet Horizon Technology

```
  v
  Modo demo: usuario simulado, datos en localStorage
```

2. Gestion de Jugadores

Crear jugador

```
/players/new ──> Formulario:
  - Nombre, edad, posicion, pie dominante
  - Altura, peso, numero de camiseta
  - Color de equipo, nivel competitivo
  - 6 metricas (speed, technique, vision, stamina, shooting, defending)
     |
     v
  playerService.create() ──> localStorage
     |
     v
  VSI calculado automaticamente (formula ponderada)
     |
     v
  syncQueue.enqueue() ──> Push a Supabase (si online)
```

Ver perfil

```
/player/:id ──> PlayerProfile.tsx
  |
  +──> VSI Gauge (score circular)
  +──> Radar Chart (6 metricas)
  +──> Banner PHV (si calculado)
  +──> Metricas Detalladas (barras)
  +──> VSI History Chart (evolucion temporal)
  +──> Analisis Avanzado:
  |    +──> TruthFilter (VSI ajustado)
  |    +──> UBI (indice de sesgo)
```

© 2026 Prophet Horizon Technology

```
|    +──> Dominant Features (estilo de juego)
|    +──> VAEP (valor por evento)
+──> Videos vinculados (grid)
+──> Link: Perfil de Rol Tactico
+──> Link: VITAS Intelligence
+──> Acciones: Editar | PDF | Eliminar
```

Calcular PHV

CODE

```
PlayerProfile ──> Click "Calcular PHV"
  |
  v
  POST /api/agents/phv-calculator
  {playerId, age, height, weight, gender}
     |
     v
  Formula Mirwald ──> phvCategory + phvOffset
     |
     v
  Player actualizado ──> VSI recalculado con ajuste PHV
  Notificacion: "PHV calculado: Tardio (offset +1.2)"
```

3. Pipeline de Video Analysis (VitasLab)

CODE

```
/lab ──> VitasLab.tsx

PASO 1: SUBIR VIDEO
  |
  +──> VideoUpload component
  |    |
  |    v
  |    POST /api/upload/video-init ──> Bunny Stream
  |    Returns: {uploadUrl, videoId, accessKey}
  |    |
  |    v
```

© 2026 Prophet Horizon Technology

```
  |    PUT directo a Bunny CDN (XHR con progress %)
  |    |
  |    v
  |    Poll /api/videos/:id/status cada 5s
  |    Espera: encodeProgress = 100%
  |    |
  |    v
  |    Video "ready" en galeria

PASO 2: SELECCIONAR VIDEO + JUGADOR
  |
  +──> Elegir video de galeria (o subir nuevo)
  +──> Elegir jugador del dropdown
  +──> Elegir modo de analisis:
  |    - "all": Todos los jugadores
  |    - "click": Click para seleccionar
  |    - "team": Comparacion de equipos
  |    - "player": Jugador especifico (camiseta + posicion)
  |
  v
  Video se vincula al jugador: VideoService.save({...video, playerId})

PASO 3: CALIBRACION (Opcional)
  |
  +──> Preset de perspectiva:
  |    - Vista Lateral
  |    - Vista Aerea
  |    - Vista Tribuna
  |    (o manual: click 4 esquinas del campo)
  |
  v
  computeHomography(4 puntos) ──> Matriz H 3x3
  Transforma: pixel (x,y) ──> campo (metros)

PASO 4: TRACKING YOLO (Opcional)
  |
  +──> Boton "Iniciar Tracking"
  |    |
  |    v
  |    trackingWorker.ts (Web Worker)
  |    YOLO v8/v11 ONNX en navegador
```

© 2026 Prophet Horizon Technology

```
  |    |
  |    v
  |    Frame-by-frame (25 FPS):
  |    Deteccion ──> Tracker (Kalman) ──> Pose Analysis
  |    |
  |    v
  |    PhysicalMetrics generadas:
  |    maxSpeed, distance, sprints, scans, duels
  |    |
  |    v
  |    TrackingMetricsPanel (tiempo real)
  |    + PlayerHeatmap
  |    + VoronoiOverlay (si ≥3 tracks + toggle ON)

PASO 5: ANALISIS IA
  |
  +──> Boton "Iniciar Analisis"
  |    |
  |    v
  |    Extrae 8 keyframes del video
  |    |
  |    v
  |    (Opcional) POST /api/agents/video-observation (Gemini)
  |    Observaciones detalladas + event counts
  |    |
  |    v
  |    POST /api/agents/video-intelligence (Claude Sonnet)
  |    {playerContext, keyframes, metrics, similarity, gemini}
  |    |
  |    v
  |    SSE Stream:
  |    [progress 10%] ──> [progress 40%] ──> [progress 80%] ──> [complete]
  |    |
  |    v
  |    Reporte renderizado:
  |    - Estado Actual (6 dimensiones)
  |    - ADN Futbolistico
  |    - Jugador Referencia (pro similar)
  |    - Proyeccion de Carrera (3 escenarios)
  |    - Plan de Desarrollo (6/18 meses + pilares)
  |    - Ejercicios Recomendados (RAG)
```

© 2026 Prophet Horizon Technology

```
|     - Riesgos Identificados
|     |
|     v
|     Guardado: Supabase player_analyses + localStorage
```

4. Tracking en Tiempo Real (YOLO)

CODE

```
Video reproduciendose en VitasLab
  |
  v
  FrameExtractor: captura frames a 8 FPS
  |
  v
  trackingWorker (Web Worker):
  +──> ONNX Runtime: YOLOv8n-pose inference
  |    Input: frame 640x640
  |    Output: [bbox, confidence, 17 keypoints] por deteccion
  |
  +──> Tracker (Hungarian Algorithm):
  |    Asigna IDs persistentes a detecciones
  |    Kalman filter para suavizar trayectorias
  |
  +──> Homography Transform:
  |    pixel (x,y) ──> campo (fx, fy) en metros
  |
  +──> PoseAnalyzer:
  |    +──> Scan Events (rotacion cabeza)
  |    +──> Duel Detection (proximidad entre jugadores)
  |
  +──> VoronoiCompute:
  |    d3-delaunay ──> poligonos por jugador
  |    Area m2 por zona
  |
  v
  UI actualizada por frame:
  +──> TrackingMetricsPanel: velocidad, distancia, sprints
```

© 2026 Prophet Horizon Technology

```
  +──> PlayerHeatmap: mapa de calor del campo
  +──> VoronoiOverlay: poligonos de control territorial
```

5. Reporte de Inteligencia (Claude AI)

CODE

```
POST /api/agents/video-intelligence
  |
  v
  Construir prompt:
  +──> Context del jugador (nombre, edad, VSI, PHV, posicion)
  +──> 8 keyframes en base64 (o URLs de Bunny)
  +──> Metricas fisicas YOLO (si disponibles)
  +──> Matches de similitud (pro player comparisons)
  +──> Observaciones Gemini (si disponibles)
  +──> Instrucciones de formato JSON
  |
  v
  Claude Sonnet (vision):
  POST https://api.anthropic.com/v1/messages
  model: claude-3-5-sonnet-20241022
  max_tokens: 4096
  |
  v
  SSE Stream al cliente:
  event: progress (10%, 30%, 60%, 80%)
  event: complete (JSON del reporte)
  event: error (si falla)
  |
  v
  Output: VideoIntelligenceOutput
  +──> estadoActual (6 dimensiones con score + observacion)
  +──> adnFutbolistico (ADN descriptivo + caracteristicas)
  +──> jugadorReferencia (nombre pro + club + posicion)
  +──> proyeccionCarrera (optimista/realista/pesimista)
  +──> planDesarrollo (objetivos 6m/18m + pilares + acciones)
  +──> recomendaciones (para entrenador + padres)
```

© 2026 Prophet Horizon Technology

6. Comparacion con Profesionales

CODE

```
Analisis de Similarity (automatico durante intelligence pipeline):
  |
  v
  Seleccionar metricas del jugador juvenil:
  [speed, technique, vision, stamina, shooting, defending]
  |
  v
  Mapear a vector EA FC25:
  [pace, dribbling, passing, physic, shooting, defending]
  |
  v
  Para cada uno de 290+ pros:
  +──> Mode 1 (Age-Adjusted): de-envejecer pro a edad del juvenil
  |    usando curvas de desarrollo
  +──> Cosine Similarity del vector 6D
  +──> Bonus/penalizacion por posicion (+5% / -15%)
  |
  v
  Ranking: Top 5 matches
  +──> bestMatch (similitud mas alta)
  +──> avgScore (promedio top 5)
  +──> dominantGroup (posicion mas comun)
  |
  v
  Clasificacion:
  92+ = "Clon" | 85+ = "Muy similar" | 75+ = "Similar" | 60+ = "Referencia"

  Herramienta de comparacion manual:
  /compare ──> PlayerComparison.tsx
  +──> Seleccionar 2-4 jugadores
  +──> Vista lado a lado: Radar, VSI, metricas, percentiles
```

7. Perfil de Rol Tactico

© 2026 Prophet Horizon Technology

CODE

```
/players/:id/role-profile ──> RoleProfile.tsx
  |
  v
  POST /api/agents/role-profile
  {playerData, metrics, phv, matchEvents}
  |
  v
  Claude AI genera perfil:
  +──> Identidad dominante (ofensivo/defensivo/tecnico/fisico/mixto)
  +──> Top 5 posiciones (GK, RB, CB, LB, DM, CM, AM, RW, LW, ST)
  |    Cada una: probabilidad, score, confianza
  +──> Top 5 arquetipos (20 posibles)
  |    Cada uno: score, confianza, estabilidad
  +──> 3 capacidades (tactical/technical/physical)
  |    Con proyecciones: actual, +6m, +18m, +36m
  +──> Fortalezas y riesgos
  |
  v
  Render en UI:
  +──> Card principal con identidad + arquetipos
  +──> Radar de posiciones
  +──> Barras de capacidades con timeline
  +──> Sample Tier (Bronze/Silver/Gold/Platinum)

  Sub-flujos:
  /role-profile/compare ──> Comparar perfiles de rol entre jugadores
  /role-profile/audit ──> Historial de cambios en el perfil
```

8. Registro de Eventos de Partido

CODE

```
PlayerProfile ──> Seccion VAEP ──> Boton "LOG"
  |
  v
  Sheet (bottom drawer):
  +──> Tipo de accion: pass|shot|dribble|tackle|press|cross|header
  +──> Resultado: success | fail
```

© 2026 Prophet Horizon Technology

```
  +──> Minuto: 1-120
  +──> Zona: offensive | middle | defensive
  |
  v
  matchEventsService.create() ──> localStorage
  |
  v
  VAEP recalculado automaticamente:
  VAEPService.calculateFromEvents(events, minutesPlayed)
  |
  v
  UI actualizada:
  +──> VAEP Total (xG acumulado)
  +──> VAEP/90 (normalizado)
  +──> Top 5 acciones (mayor impacto)
  +──> Contador de eventos
```

9. Dashboard y Monitoreo

CODE

```
/pulse ──> Dashboard.tsx

Secciones:
+──> Stats Cards (4):
|    - Jugadores Activos (total en academia)
|    - Drills Completados (historico)
|    - VSI Promedio (indice academia)
|    - Talentos Ocultos (PHV tardio + VSI < 65)
|
+──> Quick Access (4 botones):
|    - Master Dashboard
|    - VITAS.LAB
|    - Comparison Tool
|    - Director / Settings
|
+──> Partidos (LiveMatchCard):
|    Datos locales de matches programados
|
```

© 2026 Prophet Horizon Technology

```
+──> Fixtures en Vivo (LiveFixtures):
|    Football-Data.org API
|    Auto-refresh cada 60s si hay partidos en juego
|
+──> Talentos en Tendencia:
     Top jugadores con tendencia "up"
     Boton: Agregar nuevo jugador
```

10. Master Dashboard (Academia)

CODE

```
/master ──> MasterDashboard.tsx

+──> Busqueda de jugadores (nombre, posicion)
+──> Filtros: VSI tier, PHV category, academia
+──> Tabla de jugadores:
|    - Nombre, posicion, edad
|    - VSI actual + tendencia
|    - Badge de sesgo:
|      LOW (verde): madurador tardio, subestimado
|      MED (amarillo): factores mixtos
|      HIGH (rojo): madurador precoz, posible sobreestimacion
|
+──> Quick stats:
|    - Total jugadores
|    - VSI promedio academia
|    - Talentos ocultos detectados
|
+──> Click en jugador ──> /player/:id
```

11. Director Dashboard

© 2026 Prophet Horizon Technology

CODE

```
/director ──> DirectorDashboard.tsx (solo plan Club)

+──> Uso mensual:
|    - Analisis usados / limite del plan
|    - Jugadores agregados / limite
|    - Horas de video subidas
|
+──> Jugadores mas activos (este mes)
|
+──> Alertas:
|    - Uso alto (cerca del limite)
|    - Rendimiento bajo (VSI < 50)
|
+──> Detalles del plan:
     Plan actual, fecha renovacion, upgrade
```

12. Billing y Suscripciones

CODE

```
/billing ──> BillingPage.tsx

Planes:
+──> Free: 5 jugadores, 3 analisis/mes
+──> Pro ($19/mes): 25 jugadores, 20 analisis, VAEP, PDF, Push
+──> Club ($79/mes): Ilimitado, roles, director dashboard

Flujo de compra:
  Click "Suscribir" ──>
  POST /api/stripe/checkout {priceId, email}
  |
  v
  Redirect a Stripe Checkout UI
  |
  v
  Usuario paga ──>
  Stripe webhook: POST /api/stripe/webhook
  |
```

© 2026 Prophet Horizon Technology

```
    v
    Actualiza tabla `subscriptions` en Supabase
    |
    v
    syncFromSupabase() ──> Plan actualizado en app

  Portal de gestion:
    Click "Gestionar suscripcion" ──>
    GET /api/stripe/portal ──>
    Redirect a Stripe Customer Portal
```

13. Notificaciones Push

CODE

```
  /settings ──> Toggle "Notificaciones Push" ON
    |
    v
    PushNotificationService.requestPermission()
    |
    v
    (Si concedido) PushNotificationService.subscribe()
    POST /api/notifications/subscribe
    {endpoint, keys.p256dh, keys.auth}
    |
    v
    Guardado en tabla push_subscriptions

    Preferencias granulares:
    +──> Alerta de rendimiento bajo (VSI < 50)
    +──> Recordatorio de inactividad (30+ dias sin actualizar)
    +──> Alerta de limite de plan
    +──> Analisis completado

    Cron diario (09:00 UTC):
    Vercel Cron ──> GET /api/notifications/cron
    |
    v
    Para cada usuario suscrito:
```

© 2026 Prophet Horizon Technology

```
  +──> Check VSI < 50 en jugadores ──> enviar push
  +──> Check inactividad > 30 dias ──> enviar push
  +──> Check uso ≥ 90% del plan ──> enviar push
```

14. Knowledge Base (RAG)

CODE

```
Busqueda (en reporte VitasLab o standalone):
  |
  v
  KnowledgeSearch component
  Input: "ejercicios de pase"
  |
  v
  POST /api/rag/query
  {query, category?, limit}
  |
  v
  Server:
  1. Genera embedding del query (Voyage AI / Gemini)
  2. match_knowledge(embedding, threshold=0.6, limit=5) [vector search]
  3. Fallback: search_knowledge_text(query) [FTS espanol]
  |
  v
  Resultados:
  [{ content, category, similarity, metadata }]

  Ingesta:
  POST /api/rag/ingest {content, category, metadata}
  POST /api/rag/seed ──> Carga 32 drills iniciales

  Uso en pipeline:
  VitasLab reporte ──> seccion "Ejercicios Recomendados"
  Query automatico basado en areas de desarrollo del jugador
```

© 2026 Prophet Horizon Technology

15. Sincronizacion y Offline

CODE

```
App inicia (online):
  |
  v
  useSupabaseSync():
  1. Pull delta: registros con updated_at > last_synced_at
  2. Actualizar localStorage
  3. Registrar last_synced_at

  Mutacion (crear/editar jugador):
  |
  +──> (Online) ──> Push directo a Supabase
  |
  +──> (Offline) ──> syncQueueService.enqueue()
       {action, entity, entityId, data, timestamp}
       |
       v
       Almacenado en localStorage (persiste tras refresh)

  Reconexion:
  navigator.onLine event ──>
  processQueue() (FIFO):
  |
  v
  Para cada item en cola:
  +──> Push a Supabase
  +──> Si exito: dequeue()
  +──> Si fallo: retries++ (max 5, luego pruning)

  Conflictos:
  Resolucion: Last-Write-Wins
  Compara updated_at local vs remoto
  El mas reciente prevalece

  Indicador UI:
  +──> Synced (check verde)
  +──> Syncing (spinner)
```

© 2026 Prophet Horizon Technology

```
  +──> N cambios pendientes (badge)
  +──> Offline (icono rojo)
```

16. Exportacion PDF

```
PlayerProfile ──> Boton "Exportar PDF"
  |
  v
  PDFService.exportPlayerReport(playerId)
  |
  v
  Navega a /report/:id (PlayerReportPrint.tsx)
  |
  v
  Renderiza version print-friendly:
  +──> Header con logo VITAS
  +──> Datos del jugador
  +──> Radar chart
  +──> VSI + PHV + TruthFilter
  +──> Metricas detalladas
  |
  v
  html2canvas ──> Captura DOM como imagen
  |
  v
  Descarga como PDF (o Ctrl+P nativo)

  Reporte de analisis:
  /analysis-report/:id ──> AnalysisReportPrint.tsx
  +──> 6 dimensiones
  +──> Pro player comparison
  +──> Proyeccion de carrera
  +──> Plan de desarrollo
```

© 2026 Prophet Horizon Technology

Diagrama de Arquitectura General

CODE

```
                  +------------------+
                  |    Usuario PWA   |
                  +--------+---------+
                           |
                  +--------v---------+
                  |   React 18 + Vite |
                  |   (Puerto 5200)   |
                  +--------+---------+
                           |
          +--------------+--------------+
          |              |              |
+--------v---+  +-------v------+ +----v-------+
| localStorage|  | Supabase    | | Vercel API |
| (offline)   |  | (PostgreSQL) | | (Edge/Node)|
+-------------+  | + Auth + RLS| +----+-------+
                  +-------------+      |
                              +-------+--------+
                              |       |        |
                       +-----v+ +---v---+ +--v------+
                       |Claude | |Gemini | |Bunny CDN|
                       |Sonnet | |Vision | |Stream   |
                       +-------+ +-------+ +---------+
                              |
                       +-----v------+
                       | Stripe     |
                       | Payments   |
                       +------------+
```

Nota: Todos los flujos son offline-first. Si Supabase no esta configurado, la app funciona completa con localStorage. Los datos se sincronizan automaticamente cuando se detecta conexion.

© 2026 Prophet Horizon Technology

VITAS PLATFORM · BUILD 2.1.0 · © 2026 PROPHET HORIZON TECHNOLOGY